PRISMSPACE - INTELLIGENT MULTI-AGENT PLATFORM



Dept. of Computer Science & Engineering
College of Engineering, Karunagappally

Project Group 01

NOBIN SIJO	KNP23CS082
PRANAV P	KNP23CS085
PRASANTH P	KNP23CS086
SANKAR NARAYAN S	KNP23CS094

Project Coordinator & Guide
Dr. Leena Silvoster
Associate Professor
College of Engineering, Karunagappally

01. Introduction: PrismSpace Web Platform

PrismSpace is an interactive, **web-based multi-agent intelligence platform** and workspace website that allows users to orchestrate autonomous AI swarms directly from modern web browsers. It converts natural-language user goals into dependency-aware **Directed Acyclic Graphs (DAGs)** for parallel sub-task execution. Built as a responsive web application (Next.js 15, React 19) accessible from any browser with zero local setup, it features an interactive visual DAG swarm visualizer with real-time token and step streaming. The platform pairs Groq, OpenAI, and NVIDIA NIM with standardized tool connectivity via Model Context Protocol (MCP), while storing all session state locally in client-side IndexedDB (`PrismDB`) with human-in-the-loop approval gates for privacy and safety.

Problem Statement

Challenges in current AI-agent and workflow systems

02. Problem Statement: Core Challenges in AI Agent Systems

1. **Monolithic LLM Bottleneck & High Cost:** Traditional platforms route all planning, reasoning, and tool use through a single massive LLM, causing severe prompt bloat, high latency, and $3\times$ higher inference expenses.
   ➔ **PrismSpace Solution:** Dynamically routes sub-tasks to cost- and latency-optimal models (Groq for rapid inference, OpenAI/NIM for complex reasoning).

2. **Sequential & Rigid Task Execution:** Existing agent chains run strictly in linear sequence; independent sub-tasks wait needlessly in line, and a single step crash causes total pipeline collapse.
   ➔ **PrismSpace Solution:** Deconstructs goals into dynamic Directed Acyclic Graphs (DAGs) to execute independent tasks in parallel ($K \leq 3$) with isolated fault recovery.

3. **Fragmented Tool Silos & Black-Box Runs:** Proprietary integrations isolate essential productivity tools (Gmail, Drive, Terminal, Local Files), while users remain blind during long runs with no status logs.
   ➔ **PrismSpace Solution:** Standardizes all tool connections via Model Context Protocol (MCP) paired with real-time execution telemetry and progress logs.

4. **Safety Hazards & Cloud Data Lock-in:** Autonomous agents execute irreversible actions (sending emails, modifying files) without oversight, while cloud databases expose private user data.
   ➔ **PrismSpace Solution:** Enforces Human-in-the-Loop (HITL) approval gates, ORPO alignment, and zero-latency local browser database storage (`PrismDB`).

03. Comparative Analysis: Existing Platforms vs. PrismSpace

Simple comparison: Comparing PrismSpace with conversational AI, autonomous-agent, and multi-agent platforms.

Capability Dimension	ChatGPT Web (OpenAI)	AutoGPT (Autonomous Loop)	CrewAI (Multi-Agent Swarm)	PrismSpace (Ours) Local Multi-Agent Platform
Execution Topology	One conversation at a time	Repeats steps automatically; may not stop	Agents follow a fixed sequence	**DAG (Directed Acyclic Graph) Scheduling:** Runs dependent tasks in parallel, with up to three workers
Model Strategy & Routing	Mostly one provider and model	Usually one model with custom scripts	Model selected for each agent role	**Multi-Model Router:** Selects a suitable provider using task needs, speed, and cost
Tool Connectivity Protocol	Built-in plugins and function calls	Custom scripts and shell commands	Tool wrappers with fixed formats	**MCP (Model Context Protocol):** A common interface for tools such as Gmail, Drive, files, and web services
Observability & Streaming	Shows answer text; internal steps are hidden	Terminal logs only	Text traces from agent execution	**Live Run Status + Workflow Graph:** Shows task progress and each node's state
Safety & Quality Control	Provider filters; no user approval step	Fixed limits; limited output checking	Agent reviews its own output	**HITL (Human-in-the-Loop) + ORPO (Odds Ratio Preference Optimization):** Pauses risky tasks and ranks preferred outputs
Persistence & Privacy	Data stored in the provider's cloud	Local files without a structured database	Cloud or temporary in-memory state	**Local-First PrismDB:** Stores sessions and files in the browser's IndexedDB database

PrismSpace summary: It combines dependency-aware task planning, shared tool connections, provider selection, visible progress, and local browser storage in one workspace.

Project Objectives

Goals and expected outcomes of the PrismSpace platform

04. Project Objectives

1. **Context-Aware Task Planning:** Convert everyday user goals (communication, file management, daily schedules) and technical requests into structured Directed Acyclic Graphs (DAGs) for organized, dependency-safe execution.

2. **Intelligent Model & Agent Routing:** Dynamically route each sub-task to the most suitable Large Language Model (LLM) backend and dispatch specialized, context-aware agents in parallel.

3. **Practical Tool Execution via MCP:** Connect agents to everyday productivity tools and development utilities (Gmail, Google Drive, Filesystem, Terminal CLI, GitHub) through the Model Context Protocol (MCP), accompanied by live Server-Sent Events (SSE) updates.

4. **Safety Verification & Local Persistence:** Validate outputs with human approval checkpoints and Odds Ratio Preference Optimization (ORPO), storing results reactively in local IndexedDB (Indexed Database: `PrismDB`).

05. Methodology: Development Approach

- **Idea Formulation:** Identify the limitations of single-agent and sequential workflows, then define PrismSpace as a browser-based multi-agent workspace.
- **Literature and Problem Analysis:** Review multi-agent coordination, RAG, workflow scheduling, preference alignment, and human-in-the-loop methods to establish the research gap.
- **Requirements and Data Collection:** Gather representative developer tasks, tool-use traces, prompts, workspace states, and successful/failed execution outcomes for design and evaluation.
- **Data Preparation:** Clean and structure task traces, create preferred/non-preferred action pairs, and extract latency, cost, quality, and safety features.
- **Architecture Design:** Implement layered React, FastAPI, model, MCP-tool, and local IndexedDB components with observable state transitions.
- **Task Decomposition Algorithm:** Convert user goals into dependency-aware Directed Acyclic Graphs (DAGs) and execute independent tasks in parallel ($K \leq 3$).
- **Agent and Model Routing:** Use intent classification, capability matching, and cost/latency-aware routing to assign tasks to specialist agents and suitable LLMs.
- **Execution and Safety Algorithms:** Combine ReAct loops, MCP JSON-RPC tools, Server-Sent Events (SSE), Human-in-the-Loop approval, ORPO preference gating, and Isolation Forest anomaly detection.
- **Implementation and Evaluation:** Integrate the modules, persist runs in `PrismDB`, and evaluate correctness, faithfulness, latency, token cost, recovery, and safety.

Algorithm

Step-by-step execution of the PrismSpace multi-agent workflow

Algorithm: Step 1 – Understand the User Prompt

Step 1: Prompt Parsing and Structured Representation

1.1 Ingest raw natural language prompt $\mathcal{T}_{\text{user}}$ via client interface and extract core objective.

1.2 Retrieve persistent session context and workspace history from browser IndexedDB (`PrismDB`):

- **1.2.1** Load prior interaction turns, agent memory checkpoints, and active user configurations.
- **1.2.2** Resolve local file system references, target project repositories, and access tokens.

1.3 Construct strongly-typed operational execution tuple:

$$T = \langle \text{Goal, Constraints, Context, } \mathcal{M}_{\text{tools}}, \Phi_{\text{out}} \rangle$$

Where: Goal: Target objective | Constraints: Safety & SLA bounds | Context: Session state in `PrismDB` | $\mathcal{M}_{\text{tools}}$: Authorized MCP tools | Φ_{out}: Deliverable schema contract.

- **1.3.1** Parse explicit operational constraints (deadlines, schema specifications, output formats).
- **1.3.2** Bind authorized Model Context Protocol (MCP) tool capabilities and environment permissions.

1.4 Perform input boundary and security screening:

- **1.4.1** Evaluate prompt against token limits and scan for prompt injection patterns.
- **1.4.2** Flag state-mutating actions requiring mandatory human-in-the-loop authorization.

Step 2: Intent Classification and Capability Check

2.1 Categorize user prompt into functional operational domains:

$$\text{Domain} \in \{\text{Productivity/Automation, Document Synthesis, Web Research, Code Engineering}\}$$

Where: Productivity: Communication & planning | Synthesis: Summarization | Research: Web search & scraping | Coding: CLI & code synthesis.

- **2.1.1** Compute dense semantic embeddings and syntactic keyword vectors for input prompt.
- **2.1.2** Route through multi-class classifier head $\hat{\mathbf{y}}_{\text{intent}}$ to determine target capability domain.

2.2 Profile required system capabilities and operational risk levels:

- **2.2.1** Determine required subsystem resources (live web search, local filesystem I/O, terminal execution).
- **2.2.2** Enforce safety categorization: Read-Only (autonomous execution) vs. State-Mutating (approval-gated).

2.3 Select optimal execution pathway:

- **2.3.1** Route single-step queries to direct, low-latency single-model inference pipeline.
- **2.3.2** Dispatch multi-step complex requests to DAG Planner agent for task decomposition.

Step 3: DAG Decomposition and Model Routing

3.1 Deconstruct objective into Directed Acyclic Graph $\mathcal{G} = (V, E)$:

- **3.1.1** Instantiate discrete task vertices $v_i \in V$ with atomic execution contracts and output targets.
- **3.1.2** **Data Dependencies as Directed Edges** ($e = \langle v_i, v_j \rangle \in E \iff \mathcal{O}_i \cap \mathcal{I}_j \neq \emptyset$): Task v_j requires input artifacts produced by v_i (e.g., synthesis requires scraped research).
- **3.1.3** **Execution Barrier:** Consumer v_j remains blocked until producer v_i delivers $\mathcal{O}_i$, guaranteeing tasks never run on missing or partial data.

3.2 Compute topological task execution schedule $\pi = \mathrm{TopologicalSort}(\mathcal{G})$:

- **3.2.1** Identify independent vertices (in-degree $= 0$) for concurrent parallel dispatch ($K \leq 3$).
- **3.2.2** Schedule downstream vertices dynamically upon upstream output event emission.

3.3 Solve multi-objective provider routing optimization for each sub-task v_i:

$$\text{Provider}^* = \arg\min_m \left(\lambda_{\text{cost}} \, \text{Cost}_m + \lambda_{\text{latency}} \, \text{Latency}_m \right) \quad \text{s.t. } \text{Quality}_m \geq \tau$$

Where: Provider*: Optimal LLM backend | m: Candidate model | $\text{Cost}_m, \text{Latency}_m$: Normalized price & delay | $\lambda_{\text{cost}}, \lambda_{\text{latency}}$: Balancing weights | τ: Quality barrier.

- **3.3.1** Route high-throughput, latency-critical nodes to LPU backends (Groq / Llama-3).
- **3.3.2** Route deep reasoning and code generation tasks to frontier backends (OpenAI GPT-4o, Claude 3.5).

Step 4: Role-Based Task Assignment and Dispatching

4.1 Query specialist agent registry and capability profiles:

$$\mathcal{A} = \{\text{Planner, Assistant, Researcher, Coder, Validator}\}$$

4.1.1 Evaluate persona system prompts, specialized instruction boundaries, and assigned MCP tools.

4.1.2 Verify agent worker availability and monitor concurrent thread utilization.

4.2 Execute capability matching algorithm for each sub-task node v_i:

$$\text{Agent}^* = \arg\max_{a \in \mathcal{A}} \ \text{CosineSimilarity}\big(\mathbf{Req}(v_i),\, \mathbf{Cap}(a)\big)$$

Where: Agent*: Assigned specialist persona | $\mathcal{A}$: Swarm agent registry | $\mathbf{Req}(v_i)$: Task requirement vector | $\mathbf{Cap}(a)$: Agent capability embedding | CosineSimilarity: Semantic alignment.

4.2.1 Match information gathering requirements (Tag $\ni$ `web_search`) to Researcher agent.

4.2.2 Match software synthesis and execution (Tag $\ni$ `python, cli`) to Coder agent.

4.2.3 Match quality assurance and policy compliance (Tag $\ni$ `audit, verify`) to Validator agent.

4.3 Manage task lifecycle states in dispatcher queue:

4.3.1 Place sub-tasks in `PENDING` state until all parent dependency inputs resolve.

4.3.2 Transition verified nodes to `READY` and dispatch execution payloads to assigned agents.

Step 5: Agent Decision Cycle and Inter-Agent Communication

5.1 Execute autonomous Reasoning-Action (ReAct) cycle for each dispatched agent:

$$\text{State } s_t \longrightarrow \textbf{Observe} \longrightarrow \textbf{Reason} \longrightarrow \textbf{Act} \longrightarrow \textbf{Reflect}$$

Where: s_t: Current execution state at turn t | **Observe**: Ingest context & parent outputs $\mathcal{O}_i$ | **Reason**: Plan via CoT | **Act**: Invoke MCP tools | **Reflect**: Self-correct on error.

- **5.1.1** **Observe:** Ingest assigned sub-task description, operational constraints, and parent inputs.
- **5.1.2** **Reason:** Formulate intermediate chain-of-thought rationale to plan next tool invocation.
- **5.1.3** **Act:** Dispatch structured JSON-RPC tool calls to external MCP servers or formulate text.
- **5.1.4** **Reflect:** Inspect execution exit codes and output schemas; initiate local correction on errors.

5.2 Transmit intermediate artifacts via DAG edge message passing:

- **5.2.1** Route validated producer artifact $\mathcal{O}_i$ directly into consumer input buffer $\mathcal{I}_j$.
- **5.2.2** Enforce strict artifact isolation to prevent cross-task context contamination and token inflation.

5.3 Synchronize shared state via local blackboard memory:

- **5.3.1** Commit global project state, conversation turns, and file handles to IndexedDB (`PrismDB`).
- **5.3.2** Dispatch event-driven signals upon node resolution to awaken dependent downstream workers.

Step 6: Specialist Execution Workflows – Planner & Assistant

6.1 Execute Planner Agent workflow (System Architect & Coordinator):

- **6.1.1** Ingest user objective string and conversation context from `PrismDB`.
- **6.1.2** Decompose objective into atomic independent and dependent sub-goals.
- **6.1.3** Construct DAG graph vertices V, dependency edges E, and capability tags.
- **6.1.4** Solve provider routing optimization and enqueue topological schedule into Dispatcher.

6.2 Execute Assistant Agent workflow (Productivity & Communication):

- **6.2.1** Retrieve user communication preferences, profiles, and reference documents from storage.
- **6.2.2** Formulate structured response drafts, templates, or executive summaries.
- **6.2.3** Interface with Google Workspace MCP tools (Gmail draft creation, Drive document retrieval).
- **6.2.4** Emit verified communication draft; enforce approval gate prior to external message transmission.

Step 7: Specialist Execution Workflows – Researcher, Coder & Validator

7.1 Execute Researcher Agent workflow (Information Retrieval & Synthesis):

- **7.1.1** Formulate optimized web search queries and invoke Browser MCP server endpoints.
- **7.1.2** Scrape target web documents, parse DOM trees, and extract relevant contextual passages.
- **7.1.3** Deduplicate findings and compile structured Markdown report with verified source citations.

7.2 Execute Coder Agent workflow (Software Synthesis & Execution):

- **7.2.1** Inspect local repository tree via Filesystem MCP and synthesize target source code files.
- **7.2.2** Execute scripts inside Sandboxed Terminal CLI and inspect exit codes and error streams.
- **7.2.3** Analyze stack traces and compiler lints to autonomously self-correct syntax and runtime bugs.

7.3 Execute Validator Agent workflow (Quality Auditing & Safety Gate):

- **7.3.1** Scan candidate outputs against safety policies, token invariants, and functional constraints.
- **7.3.2** Emit binary verification signal: **PASS** to release artifact, or **FAIL** with error diagnostics.

Step 8: Standardized Tool Execution via Model Context Protocol

8.1 Interface agents to external tools via Model Context Protocol (MCP):

- **8.1.1** Standardize client-server communication using JSON-RPC protocol specifications.
- **8.1.2** Enable dynamic tool discovery: inspect exposed tool schemas, input parameters, and return types.
- **8.1.3** Decouple agent reasoning logic from external tool implementation details.

8.2 Orchestrate multi-domain tool execution ecosystem:

- **8.2.1** **System & Dev Tools:** Local Filesystem (file I/O), Sandboxed CLI (isolated shell), GitHub API.
- **8.2.2** **Productivity Tools:** Google Workspace MCP (Gmail messaging, Google Drive document access).
- **8.2.3** **Web Intelligence:** Browser MCP for real-time document search, retrieval, and scraping.

8.3 Enforce Human-in-the-Loop (HITL) authorization policy:

- **8.3.1** Permit autonomous execution for non-destructive, read-only tool queries.
- **8.3.2** Intercept state-mutating actions (file modification, email dispatch, CLI execution) for user approval.

Step 9: Context Compaction and Shared Memory Management

9.1 Address context window constraints and token consumption limits:

9.1.1 Mitigate fixed LLM context window boundaries to eliminate token overflow and latency degradation.

9.1.2 Prevent quadratic attention computation overhead across long multi-agent interaction sessions.

9.2 Execute dynamic context compaction and artifact pruning:

$$C_{t+1} = \mathrm{Prune}\big(C_t \cup \Delta\text{Artifacts} \setminus \text{Stale Tokens}\big)$$

Where: C_{t+1}, C_t: Next and current context buffers | ΔArtifacts: Verified upstream deliverables $\mathcal{O}_i$ | Stale Tokens: Discarded scratchpad tokens & raw tool logs | $\mathrm{Prune}(\cdot)$: Heuristic eviction filter.

9.2.1 Forward only verified parent output artifacts $\mathcal{O}_i$ to dependent downstream child nodes.

9.2.2 Evict intermediate reasoning scratchpads, redundant tool telemetry, and discarded draft tokens.

9.3 Persist global execution state using client-side database (`PrismDB`):

9.3.1 Commit session turns, task graphs, and tool execution logs to browser IndexedDB via Dexie.js.

9.3.2 Ensure client data privacy, full offline execution persistence, and instant crash recovery.

Step 10: Single-Stage Preference Alignment (ORPO) and Verification

10.1 Single-stage reference-free preference alignment formulation (ORPO):

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda_{\text{OR}} \cdot \mathcal{L}_{\text{OR}} \quad \text{where} \quad \mathcal{L}_{\text{OR}} = -\log \sigma \left(\log \left[\frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)} \right] \right)$$

Where: $\mathcal{L}_{\text{ORPO}}$: Total loss | $\mathcal{L}_{\text{SFT}}$: SFT cross-entropy loss | λ_{OR}: Odds weight | $\mathcal{L}_{\text{OR}}$: Odds ratio penalty | $\text{odds}_\theta(y \mid x) = \frac{P_\theta(y|x)}{1-P_\theta(y|x)}$: Output sequence odds | y_w, y_l: Chosen / rejected responses for prompt x.

10.1.1 Jointly maximize chosen likelihood while penalizing rejected tokens ($2\times$ memory savings).

10.1.2 Supervise via UltraFeedback pairs $\mathcal{D} = \{(x, y_w, y_l)\}$, bypassing unstable RLHF loops.

10.2 Enforce runtime quality gating via preference score barrier $\mathcal{S}_{\text{ORPO}}(x, y) \geq \tau_{\text{approval}}$:

10.2.1 Audit deliverables using fine-tuned model (`reward_orpo_v2`, Qwen2.5-1.5B LoRA).

10.2.2 Score candidate outputs on instruction adherence, structural constraints, and safety guardrails.

10.3 Handle quality failures via targeted DAG re-planning:

10.3.1 Capture diagnostic error traces and schema deviations when output falls below τ_{approval}.

10.3.2 Dispatch error logs to DAG Planner for targeted branch re-planning without system abort.

Step 11: Artifact Assembly and Global State Commit

11.1 Aggregate and assemble multi-agent deliverables:

11.1.1 Harvest verified partial output artifacts $\mathcal{O}_i$ from all terminal nodes in the completed DAG.

11.1.2 Coalesce research summaries, verified code diffs, and formatted text into unified final deliverable.

11.2 Calculate end-to-end composite delivery quality metric:

$$\text{Delivery Quality } D = \text{Correctness} \times \text{Completeness} \times \text{Timeliness}$$

Where: $D \in [0, 1]$: Composite quality metric | Correctness: Schema conformance & execution pass rate | Completeness: Fulfillment of goal constraints | Timeliness: Latency penalty score.

11.2.1 Apply multiplicative scoring penalty: failure in any single dimension sharply penalizes overall quality.

11.2.2 Log quality vector to persistent performance ledger to calibrate future provider routing models.

11.3 Commit finalized session state with ACID guarantees:

11.3.1 Atomically write completed conversation records, DAG execution graphs, and tool receipts to `PrismDB`.

11.3.2 Record end-to-end telemetry: aggregate latency, per-agent token consumption, and audit receipts.

Step 12: Output Delivery and Reactive Workspace Presentation

12.1 Synthesize and package final deliverable artifacts:

- **12.1.1** Compile structured digital artifacts (Markdown reports, verified code patches, email/document drafts).
- **12.1.2** Generate comprehensive audit receipt containing agent provenance, execution duration, and tool logs.

12.2 Render presentation layer via multi-pane workspace interface:

- **12.2.1** Update interactive DAG HUD with real-time node states (`Queued` → `Running` → `Success` / `Failed`).
- **12.2.2** Mount Artifact Viewer canvas providing rich Markdown rendering, syntax highlighting, and export controls.
- **12.2.3** Present interactive approval cards for pending external actions with explicit authorization controls.

12.3 Synchronize client state reactively via local database transactions:

- **12.3.1** Commit verified state transitions and output artifacts directly into browser IndexedDB (`PrismDB`).
- **12.3.2** Trigger Dexie `useLiveQuery()` observers to update UI components reactively upon local database commits.

Project Architecture, Working & System Design

Project workflow, architecture, working process, agent lifecycle, dataset pipeline, DAG scheduling, and flow diagrams

06. PrismSpace Workflow (Figure 1.0)

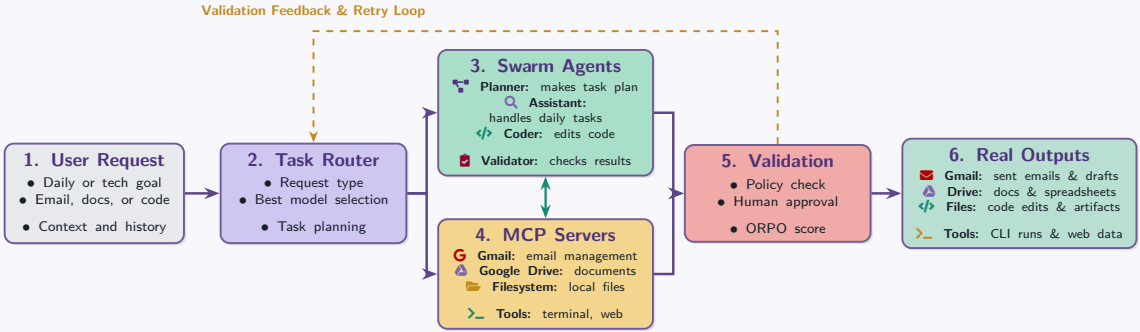
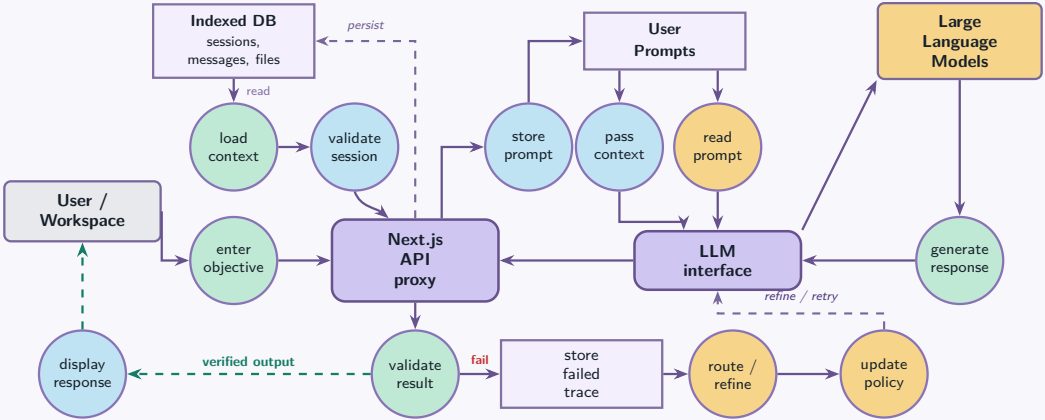


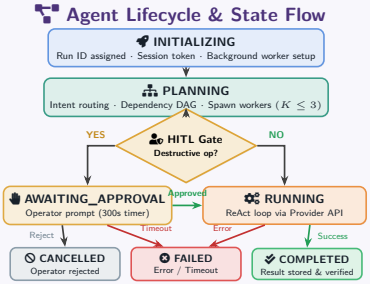
Figure 1.0: PrismSpace Workflow | **DAG** = Directed Acyclic Graph | **MCP** = Model Context Protocol | **ORPO** = Odds Ratio Preference Optimization

07. PrismSpace Data Flow Diagram

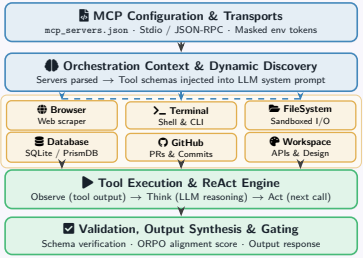


07. Agent Lifecycle, MCP Integration & Runtime Flow in PrismSpace

Agent Lifecycle & State Flow



MCP Tool Integration & Runtime Flow



Agent Lifecycle & State Machine:

- **Execution Progression:** Orchestrates transitions from `INITIALIZING` → `PLANNING` (DAG task compilation, bounded to $K \leq 3$ workers) → active `RUNNING` ReAct loop.
- **HITL Safety & Terminal States:** Pauses destructive actions in `AWAITING_APPROVAL` for human sign-off (300s timeout); guarantees deterministic resolution into `COMPLETED`, `FAILED`, or `CANCELLED`.

Model Context Protocol (MCP) Runtime:

- **Universal Open Standard:** Decouples LLMs from tools via standard JSON-RPC stdio transports, dynamically registering capabilities from `mcp_servers.json`.
- **ReAct Tool Execution:** Workers invoke sandboxed capabilities (Browser, CLI, DB) via iterative ReAct loops; outputs are validated and ORPO-scored before synthesis.

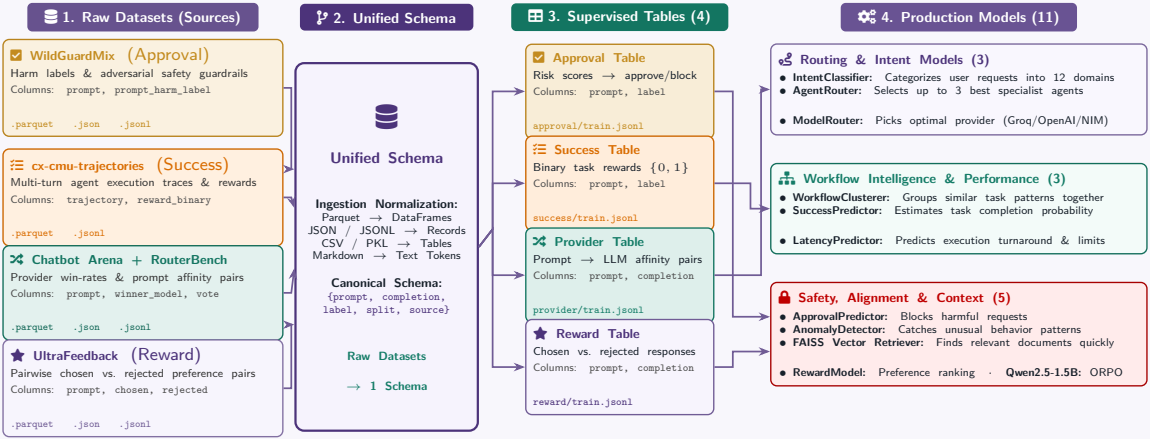
09. Model Responsibilities & Inference Flow

Intent: The **underlying goal** of a user's request. Determines *task type* (research, coding, tool use, synthesis) for routing to agents & tools.

1. Intent Classifier	**Prompt** → **Extracted features** → **classify intent** (research, tool use, coding, everyday synthesis)	**Task type** `intent_classifier.joblib`
2. Agent Router	**Intent + task features** → predict **multiple agent labels** → form the **specialist team**	**Agent selection** `agent_router.joblib`
3. Workflow Planner	**Workflow templates** → estimate **completion probability** → choose a **dependency-safe DAG plan**	**Plan reuse + success** `workflow_templates.pkl` `workflow_success_predictor.joblib`
4. Provider Routing	**Candidate task** → choose **provider/model** → compare **predicted latency** and **token cost**	**Provider + cost balance** `model_router.joblib` `latency_predictor.joblib` `cost_predictor.joblib`
5. Safety Gate	**Planned action** → score **approval risk** + **detect anomalies** → **approve, pause, or block** execution	**Risk approval & safety** `approval_predictor.joblib` `anomaly_detector.joblib`
6. Context & Quality	**Query** → **FAISS semantic retrieval** → add **context** → generate & **ORPO-align response**	**Evidence + output quality** `faiss.index` · `faiss.vectorizer.joblib` `reward_orpo_v2/adapter_model.safetensors`

10. Unified Dataset Preparation & Multi-Model Training Pipeline

Raw multi-source datasets across multiple formats are normalized into a **unified schema**, curated into **4 supervised tables**, and trained into **11 production models**.



11. How the System Works: From Request to Result

Stage 1: Ingestion & Features

- **Capture request:** Parse user input (text/voice/API).
- **Clean & normalize:** Remove noise, handle missing data with NLP preprocessing.
- **Extract features:** Tokenize into semantic embeddings using transformer models.

Stage 2: Intelligence & DAG

- **Classify intent:** ML classifier identifies task type from 12 categories (99.6% accuracy).
- **Choose agents & models:** Route sub-tasks to specialized LLM agents.
- **Build the DAG:** Create dependency graph ensuring safe execution order.

Stage 3: Execution & Storage

- **Execute:** Run up to 3 parallel tasks using Model Context Protocol (MCP) tools.
- **Track progress:** Real-time telemetry, node state transitions, and execution logs.
- **Verify & store:** ORPO quality check, persist results to PrismDB.

Building & Executing the Task Graph $\mathcal{G}(V, E)$

1. User Goal Prompt
"Analyze news+CSV, email summary"

→

2. Split into Vertices V
Modular async tasks A, B, C, D

→

3. Connect Edges E
Map output return → input args

→

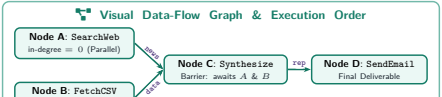
4. Topological Run
Run in-degree $= 0 \implies$ join barrier

</> Computer Science Explanation (By Concrete Example):

- **Vertices (V) = Sub-Task Functions (Async Call Units):**
  Node A: `news = await SearchWeb("Tesla news")` *[Researcher]*
  Node B: `data = await FetchCSV("metrics.csv")` *[Data Agent]*
  Node C: `rep = await Synthesize(news, data)` *[Writer Agent]*
  Node D: `await SendEmail(to="lead", body=rep)` *[Email Tool]*
- **Edges (E) = Data Pipes & Synchronization Barriers:**
  Edge $\langle A, C \rangle$ pipes return value `news`; edge $\langle B, C \rangle$ pipes `data`.
  Barrier: task C blocks until both promises A & B resolve.
- **Connection Basis (Input ← Output Matching):**
  Function C links to A & B because C's parameters require their outputs. A and B share no edge because neither depends on the other's data.
- **Execution Schedule (Topological Concurrency):**
  Tasks with in-degree $= 0$ (A, B) execute **concurrently in parallel**.

When A & B complete, C's in-degree drops to $0 \implies C$ runs $\implies D$ runs.

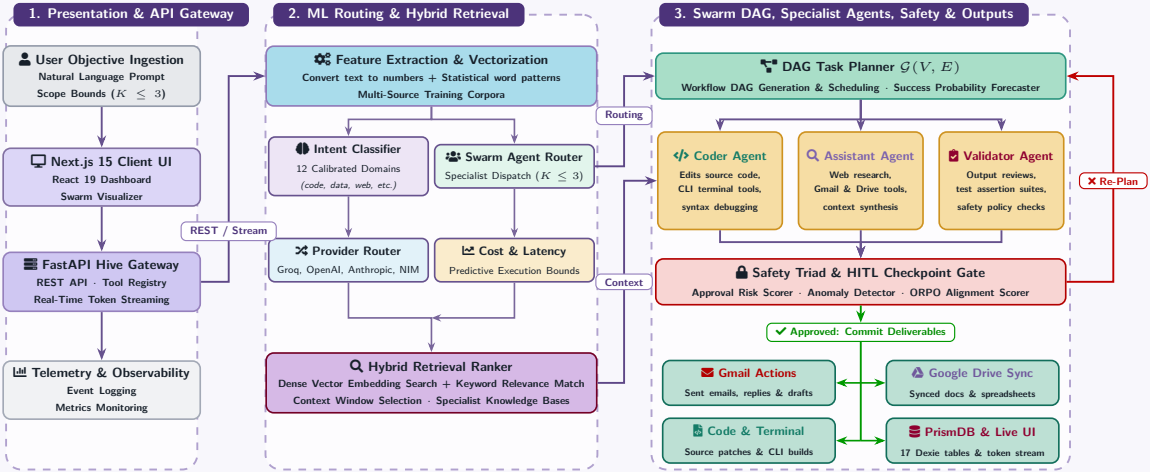
Visual Data-Flow Graph & Execution Order



Parallel Thread 1: Worker 1 executes Node A → returns `news`
Parallel Thread 2: Worker 2 executes Node B → returns `data`
Join Barrier: Node C consumes `news` + `data` → outputs `rep`

Serial Finish: Node D consumes `rep` → sends email

12. PrismSpace System Architecture & Multi-Model Swarm Workflow (Figure 1.2)



13. Flowchart: Multi-Agent Execution Flow (Figure 1.3)

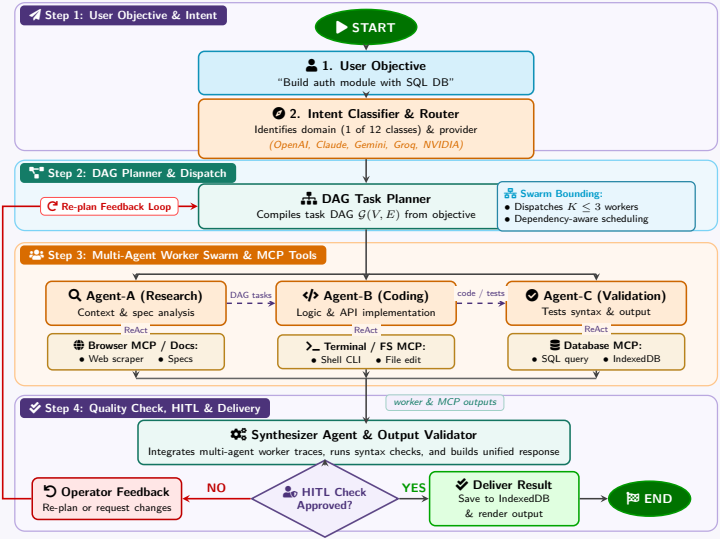


Figure 1.3: Simplified Multi-Agent Communication Flow in PrismSpace

Database Design

PrismDB schema, relationships, and entity–relationship structure

14. Database Schema

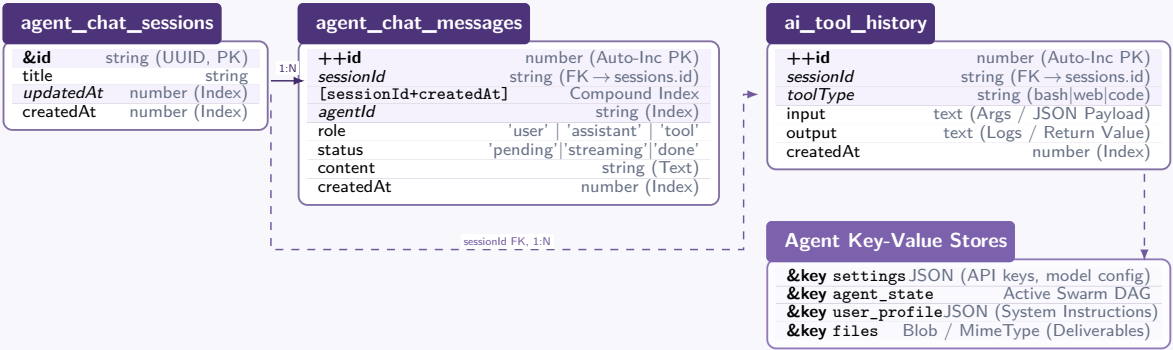
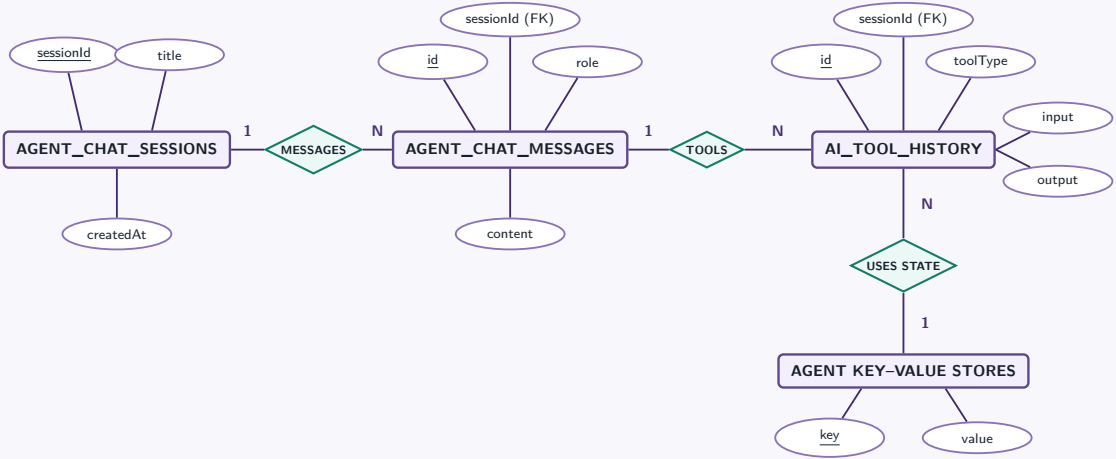


Figure 1.4: PrismDB - AI-Agent Browser IndexedDB Schema (Dexie.js v3) | **PK**: Primary Key **FK**: Foreign Key **Index**: B-tree index
`sessions.id` (Shared FK) → `messages.sessionId` & `tool_history.sessionId` | `useLiveQuery()` Reactive UI | ACID Dexie.js

15. Database ER Diagram



Project Requirements

Hardware and software needed to build and run PrismSpace

16. Hardware Requirements

- **CPU:** Multi-core processor for the Next.js client, FastAPI service, and local development tools.
- **GPU:** NVIDIA RTX 3050 or above for embedding generation, vector indexing, and model-training workloads.
- **Memory:** Minimum 16 GB RAM for running the frontend, backend, databases, and supporting services concurrently.
- **Storage:** SSD with sufficient capacity for the codebase, PrismDB data, logs, model checkpoints, and FAISS indexes.
- **Network:** Reliable internet connection for provider APIs, MCP integrations, and model downloads.
- **Security:** Secure environment-variable or secret-manager support for provider and tool credentials.

17. Software Requirements

- **Next.js 14 / React / TypeScript:** browser-based workspace for objectives, chat history, agent status, and approvals.
- **FastAPI / Python:** asynchronous backend framework for agent orchestration, REST endpoints, and runtime events.
- **Dexie.js / IndexedDB:** local ACID storage for sessions, messages, files, settings, and tool history.
- **Joblib / scikit-learn:** Python libraries for data preparation, utility functions, serialization, and general machine-learning support.
- **MCP JSON-RPC:** MCP (Model Context Protocol) uses JSON-RPC (JavaScript Object Notation–Remote Procedure Call) messages to standardize tool names, inputs, outputs, and errors for browser, terminal, filesystem, GitHub, database, and workspace tools.
- **Provider SDKs/APIs: NVIDIA NIM** provides optimized model-serving endpoints; the **Groq SDK** supports low-latency inference; and the **Anthropic Claude SDK/API**, OpenAI-compatible APIs, and similar provider clients connect the orchestrator to different language models.

Project Implementation

Frontend, backend, and training components

18. Frontend Implementation

- **Next.js 14:** React-based framework for building the browser application and managing frontend pages.
- **React:** component library for creating reusable user-interface elements.
- **TypeScript:** typed JavaScript used for safer and more maintainable frontend development.
- **Tailwind CSS:** utility-first styling framework for responsive layouts and consistent visual design.
- **Dexie.js, IndexedDB, and dexie-react-hooks:** browser storage and reactive data-access libraries for local application data.
- **Lucide React:** icon library used for interface symbols and controls.
- **Framer Motion and GSAP (GreenSock Animation Platform):** frontend animation libraries for smooth interface transitions.

19. Backend Implementation

- **FastAPI:** Python framework for building asynchronous backend services and application endpoints.
- **Python:** primary backend language for API services, integrations, data handling, and runtime coordination.
- **Uvicorn:** lightweight **ASGI (Asynchronous Server Gateway Interface)** server used to run FastAPI. ASGI connects asynchronous Python applications to web servers and supports non-blocking concurrent requests.
- **REST (Representational State Transfer):** API style for communicating with the frontend through structured HTTP requests and responses.
- **Pydantic:** schema validation and typed data models for request, response, configuration, and tool payloads.
- **MCP (Model Context Protocol) and JSON-RPC (JavaScript Object Notation–Remote Procedure Call):** standards for structured communication between the backend and external tools.
- **Provider SDKs/APIs:** integration layer for connecting the backend to external language-model and model-serving services.

20. Model Training Modules

- **Dataset preparation:** collects, cleans, and combines task traces and preference data using **Pandas**, **NumPy**, and **PyArrow**.
- **Intent classification:** categorizes user objectives using **scikit-learn** text features, classifiers, and **XGBoost**.
- **Agent routing:** selects specialist agents using shared **scikit-learn** pipelines and **XGBoost** classification support.
- **Model and provider routing:** compares task features and provider signals using **Pandas**, **NumPy**, **scikit-learn**, and **XGBoost**.
- **Performance prediction:** estimates latency and cost with **XGBoost** regression, supported by **scikit-learn** preprocessing.
- **Safety prediction:** estimates approval needs and detects unusual patterns with **scikit-learn**, **XGBoost**, and **Isolation Forest** utilities.
- **Retrieval:** uses **FAISS (Facebook AI Similarity Search)**, **NumPy**, and optional **sentence-transformers** embeddings for context search.
- **Preference alignment:** uses **PyTorch**, **Hugging Face Transformers**, **TRL (Transformer Reinforcement Learning)**, **PEFT**, and **Datasets** for ORPO training.
- **Runtime artifacts:** uses **Joblib**, **Safetensors**, and **Hugging Face Accelerate** to save, validate, and load trained components.

Artifact: a saved output from training, such as a trained model, vector index, or configuration file, that can be reused later. **FAISS (Facebook AI Similarity Search):** a library for quickly finding similar vectors. **PEFT (Parameter-Efficient Fine-Tuning):** adapts a model by training only a small set of additional parameters.

System Design

Modules, responsibilities, and system-level organization

21. System Modules and Responsibilities

- **User and Session Module:** captures objectives, settings, conversation history, uploaded files, and session state.
- **Context and Routing Module:** classifies intent and selects the most suitable provider, model, and specialist agents.
- **DAG (Directed Acyclic Graph) Planning Module:** decomposes objectives into dependency-aware subtasks and schedules parallel work.
- **Agent Execution Module:** runs bounded **ReAct (Reasoning and Acting)** loops and invokes approved tools.
- **MCP (Model Context Protocol) Integration Module:** exposes filesystem, terminal, database, memory, and external-service tools through **JSON-RPC (JavaScript Object Notation–Remote Procedure Call)**.
- **Verification and Safety Module:** applies schema checks, **ORPO (Odds Ratio Preference Optimization)** preference gating, anomaly detection, and **HITL (Human-in-the-Loop)** approvals.
- **Persistence Module:** stores sessions, messages, artifacts, and errors in the local **PrismDB (browser IndexedDB database)**.

22. Conclusion

- **PrismSpace** converts a user's natural-language goal into smaller, organized tasks.
- Independent tasks can be executed by multiple agents in parallel.
- Standard tools are connected through the **Model Context Protocol (MCP)**.
- Human approval and local browser storage improve safety, privacy, and control.
- The platform provides a flexible foundation for future agent-based applications.

References

[1] **Cornalba F, Disselkamp C, Scassola D, Helf C (2024)** Multi-objective reward generalization: improving performance of Deep Reinforcement Learning for applications in single-asset trading. *Neural Computing and Applications*. DOI: 10.1007/s00521-023-09033-7

[2] **Schneckenreither M, Moser G (2025)** Average reward adjusted discounted reinforcement learning. *Neural Computing and Applications*. DOI: 10.1007/s00521-024-10620-5

[3] **Aina K, Ha S (2026)** Deep reinforcement learning for multi-agent coordination. *Artificial Life and Robotics*. DOI: 10.1007/s10015-025-01089-z

[4] **Rother D, Pajarinen J, Peters J, Weisswange TH (2025)** Open-ended coordination for multi-agent systems using modular open policies. *Autonomous Agents and Multi-Agent Systems*. DOI: 10.1007/s10458-025-09723-7

[5] **Liu D, Ren F, Yan J, Su G, Gu W, Kato S (2026)** Improving scalability of multi-agent deep reinforcement learning with suboptimal human knowledge. *Autonomous Agents and Multi-Agent Systems*. DOI: 10.1007/s10458-025-09729-1

[6] **Marzari L, Liu C, Donti PL, Marchesini E (2026)** ε-retraining reinforcement learning algorithms. *Autonomous Agents and Multi-Agent Systems*. DOI: 10.1007/s10458-026-09748-6

[7] **Wong A, Bäck T, Kononova AV, Plaat A (2023)** Deep multiagent reinforcement learning: challenges and directions. *Artificial Intelligence Review*. DOI: 10.1007/s10462-022-10299-x

[8] **Niu B, Luo B, Cui Y, Xu X, Zhao Y, Feng Y (2025)** HIAT: human-in-the-loop reinforcement learning with auxiliary task. *Artificial Intelligence Review*. DOI: 10.1007/s10462-025-11260-4

References

[9] **Lee J (2025)** InstructPatentGPT: training patent language models to follow instructions with human feedback. *Artificial Intelligence and Law*. DOI: 10.1007/s10506-024-09401-1

[10] **Zhuang S, Qiao Z, Zuccon G (2023)** Reinforcement online learning to rank with unbiased reward shaping. *Information Retrieval Journal*. DOI: 10.1007/s10791-022-09413-y

[11] **Ghazal TM, Awouda AE, Hasan MK, Rahman AHA, Islam S, Saeed RA, Elshafie H, Iqbal A, Hussein AH (2025)** Knowledge Learning for Securing Workflow Scheduling Algorithm in Mobile Edge Computing. *Mobile Networks and Applications*. DOI: 10.1007/s11036-025-02465-6

[12] **Barredo P, Puente J (2025)** Energy-aware cooperative multi-fitness evolutionary algorithm for workflow scheduling in cloud computing. *Natural Computing*. DOI: 10.1007/s11047-025-10023-y

[13] **Heirati A, Fartash M, Khalily-Dermany M (2025)** Optimized Task Scheduling in Fog-Cloud Computing Using Hybrid Deep Learning and Metaheuristic Algorithms. *Neural Processing Letters*. DOI: 10.1007/s11063-025-11819-w

[14] **Abdi S, Ashjaei M, Mubeen S (2024)** Cost-aware workflow offloading in edge-cloud computing using a genetic algorithm. *The Journal of Supercomputing*. DOI: 10.1007/s11227-024-06341-0

[15] **Xia Y, Zhu J, Zhu L (2023)** Dynamic role discovery and assignment in multi-agent task decomposition. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-023-01071-x

[16] **Li J, Xing L, Zhong W, Cai Z, Hou F (2023)** Decision variable contribution based adaptive mechanism for evolutionary multi-objective cloud workflow scheduling. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-023-01137-w

References

[17] **Yu X, Lin Y, Wang X, Han S, Lv K (2024)** GHQ: grouped hybrid Q-learning for cooperative heterogeneous multi-agent reinforcement learning. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-024-01415-1

[18] **Xu C, Wang J, Zhu X, Yue Y, Zhou W, Liang Z, Wojtczak D (2024)** Decentralized multi-agent cooperation via adaptive partner modeling. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-024-01421-3

[19] **Vyhmeister E, Castañé GG, Östberg PO, Thevenin S (2023)** A responsible AI framework: pipeline contextualisation. *AI and Ethics*. DOI: 10.1007/s43681-022-00154-8

[20] **Cavalcante Siebert L, Lupetti ML, Aizenberg E, Beckers N, Zgonnikov A, Veluwenkamp H, Abbink D, Giaccardi E, Houben GJ, Jonker CM, van den Hoven J, Forster D, Lagendijk RL (2023)** Meaningful human control: actionable properties for AI system development. *AI and Ethics*. DOI: 10.1007/s43681-022-00167-3

[21] **Yue Q, Tian L, Feng X, Yuan Z, Wei S, Xia J, Chen B, Song X, Yao Y (2026)** CPN-HRL: a hierarchical deep reinforcement learning approach for priority-aware task scheduling in CPN enabled by cloud-edge-end environments. *Journal of King Saud University Computer and Information Sciences*. DOI: 10.1007/s44443-026-00690-x

[22] **Skirzyński J, Jain Y, Lieder F (2023)** Automatic discovery and description of human planning strategies. *Behavior Research Methods*. DOI: 10.3758/s13428-023-02062-z

[23] **Zhang S, Zhao Z, Liu C, Qin S (2023)** Data-intensive workflow scheduling strategy based on deep reinforcement learning in multi-clouds. *Journal of Cloud Computing*. DOI: 10.1186/s13677-023-00504-9

[24] **Wang Z, Chen S, Bai L, Gao J, Tao J, Bond RR, Mulvenna MD (2023)** Reinforcement learning based task scheduling for environmentally sustainable federated cloud computing. *Journal of Cloud Computing*. DOI: 10.1186/s13677-023-00553-0

[25] **Subashree S, Rajakumaran M, Pushpa G, Senthilkumar S (2025)** Hybrid dolphin swarm sparrow search optimization based multi-objective cloud workflow scheduling. *Journal of Cloud Computing*. DOI: 10.1186/s13677-025-00828-8

Thank You!

Nobin Sijo | Pranav P | Prasanth P | Sankar Narayan S

Department of Computer Science & Engineering
College of Engineering, Karunagappally